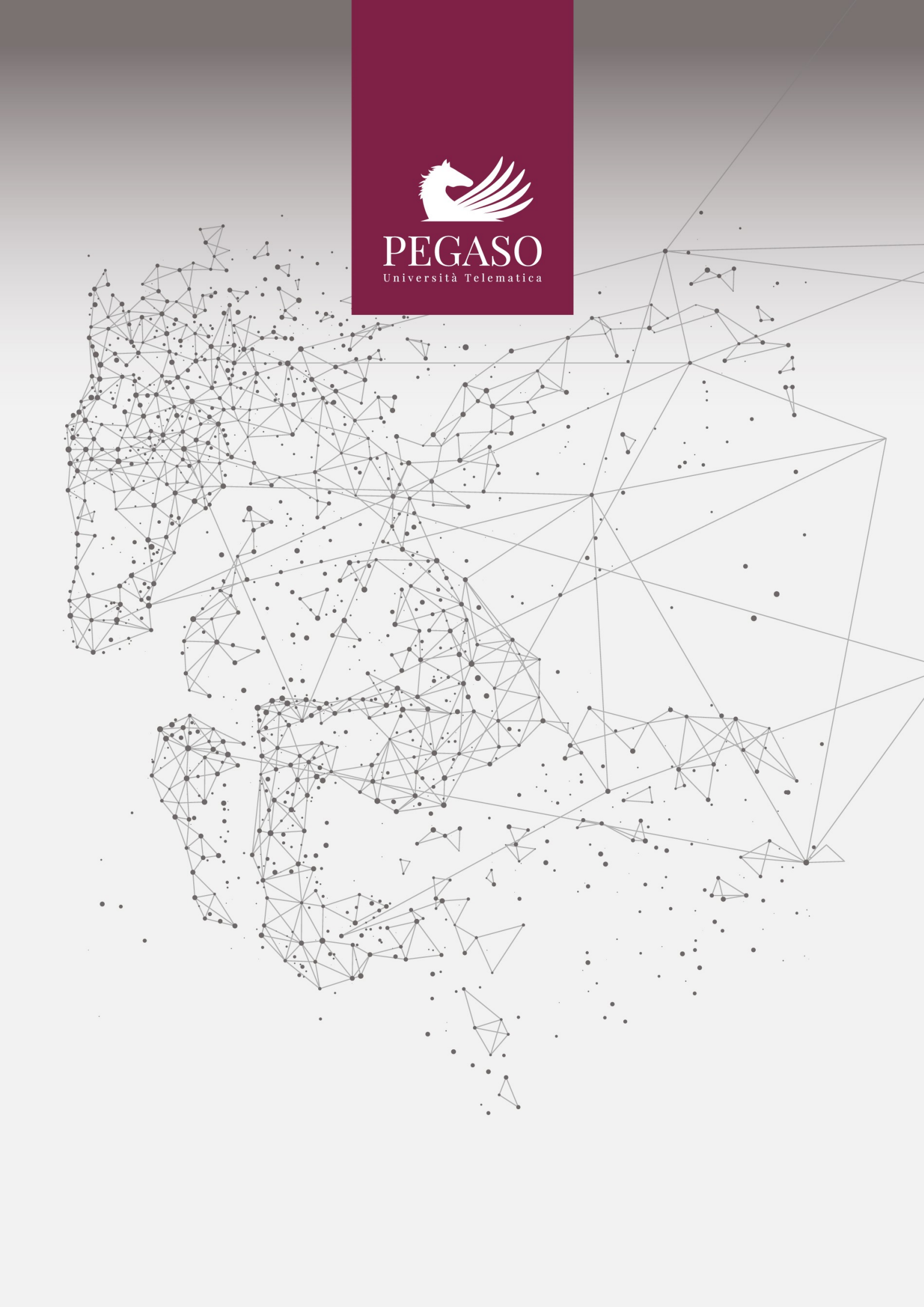

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. JS HTML DOM 3
2. JAVASCRIPT HTML DOM 7
3. EVENTI 10
BIBLIOGRAFIA E SITOGRAFIA 14

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. JS HTML DOM

Il JavaScript HTML DOM (Document Object Model) è un'interfaccia di programmazione delle applicazioni (API) che consente di manipolare gli elementi HTML e XML di una pagina web attraverso il linguaggio di programmazione JavaScript. In pratica, il DOM rappresenta l'albero di struttura della pagina web e ogni elemento è rappresentato da un oggetto del DOM.

Grazie al JavaScript HTML DOM, è possibile accedere, creare e modificare gli elementi di una pagina web dinamicamente, senza dover ricaricare la pagina stessa: questo consente di creare interazioni dinamiche con gli utenti, animazioni, applicazioni web interattive e molto altro ancora.

Per accedere agli elementi del DOM, si utilizzano i metodi come:

- getElementById
- getElementsByClassName
- getElementsByTagName
- querySelector
- querySelectorAll

Una volta acceduto all'elemento desiderato, è possibile manipolarlo modificando le sue proprietà, come ad esempio innerHTML, textContent, style e classList.

Inoltre, è possibile aggiungere eventi a un elemento utilizzando il metodo addEventListener e creare nuovi elementi HTML utilizzando il metodo createElement. Grazie a queste funzionalità, è possibile creare pagine web dinamiche e interattive che rispondono alle azioni degli utenti.

Analizziamo i metodi per accedere agli elementi del DOM:

- **getElementById()**: questo metodo restituisce l'elemento HTML che ha un attributo "id" corrispondente al valore specificato come argomento.

```
var elemento = document.getElementById("idElemento");
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **getElementsByClassName()**: questo metodo restituisce una collezione di tutti gli elementi HTML che hanno una classe CSS corrispondente al valore specificato come argomento.

```
var elementi=document.getElementsByClassName("classElem");
```

- **getElementsByTagName()**: questo metodo restituisce una collezione di tutti gli elementi HTML che hanno un tag HTML corrispondente al valore specificato come argomento.

```
var elementi = document.getElementsByTagName("tagElem");
```

- **querySelector()**: questo metodo restituisce il primo elemento HTML che corrisponde al selettore CSS specificato come argomento.

```
var elemento=document.querySelector("#idElem .classElem");
```

- **querySelectorAll()**: questo metodo restituisce una collezione di tutti gli elementi HTML che corrispondono al selettore CSS specificato come argomento.

```
var elementi=document.querySelectorAll(".classeElem");
```

Riportiamo di seguito quali sono i selettori CSS che possono essere utilizzati per poter accedere al DOM della pagina.

Selettore di tipo

Il selettore di tipo seleziona tutti gli elementi di un determinato tipo HTML; ad esempio, il selettore di tipo "p" selezionerà tutti gli elementi di paragrafo nella pagina. La sintassi è il nome del tipo di elemento, ad esempio "p" o "h1".

Selettore di classe

Il selettore di classe seleziona tutti gli elementi che hanno una determinata classe HTML; le classi vengono definite utilizzando l'attributo "class" negli elementi HTML e in CSS vengono indicate con un punto ".". Ad esempio, il selettore ".intro" selezionerà tutti gli elementi con la classe "intro". La sintassi è il punto "." seguito dal nome della classe, ad esempio ".intro".

Selettore di ID

Il selettore di ID seleziona un singolo elemento con un determinato ID HTML; gli ID vengono definiti utilizzando l'attributo "id" negli elementi HTML e in CSS vengono indicati con un cancelletto "#". Ad

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

esempio, il selettore "#header" selezionerà l'elemento con l'ID "header". La sintassi è il cancelletto "#" seguito dal nome dell'ID, ad esempio "#header".

Selettore discendente

Il selettore discendente seleziona tutti gli elementi figli di un altro elemento; la sintassi è quella di elencare gli elementi separati da uno spazio. Ad esempio, il selettore "ul li" selezionerà tutti gli elementi "li" che sono figli di un elemento "ul".

Selettore di figlio diretto

Il selettore di figlio diretto seleziona solo gli elementi che sono figli diretti di un altro elemento; la sintassi è quella di elencare gli elementi separati da ">", ad esempio "ul > li" selezionerà solo gli elementi "li" che sono figli diretti di un elemento "ul".

Selettore di attributo

Il selettore di attributo seleziona gli elementi in base a un attributo HTML specifico. La sintassi è quella di scrivere il nome dell'attributo tra parentesi quadre, ad esempio "[href]" selezionerà tutti gli elementi che hanno un attributo "href".

Selettore di attributo con valore

Il selettore di attributo con valore seleziona gli elementi in base a un attributo HTML specifico e al suo valore. La sintassi è quella di scrivere il nome dell'attributo seguito dall'operatore di uguaglianza "=" e dal valore desiderato tra virgolette, ad esempio "[href='#']" selezionerà tutti gli elementi che hanno un attributo "href" con valore "#".

Selettore di pseudo-classe

Il selettore di pseudo-classe seleziona gli elementi in base a uno stato o una condizione specifici, come ad esempio quando l'utente posiziona il mouse sopra l'elemento o quando è stato visitato un link. La sintassi è quella di scrivere il nome della pseudo-classe preceduto da due punti, ad esempio ":hover" selezionerà l'elemento quando l'utente posiziona il mouse sopra di esso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Selettore di pseudo-elemento

Il selettore di pseudo-elemento seleziona parti specifiche di un elemento HTML, come ad esempio la prima lettera di un paragrafo o la prima riga di un blocco di testo. La sintassi è quella di scrivere il nome del pseudo-elemento preceduto da due punti, ad esempio "::first-letter" selezionerà la prima lettera di ogni elemento selezionato.

Riportiamo alcuni esempi di gestione delle proprietà di elementi del DOM usando Javascript:

```
// seleziona l'elemento con l'ID "header"
var header = document.getElementById("header");

// modifica il CSS "background-color" dell'elemento selezionato
header.style.backgroundColor = "blue";

// seleziona tutti gli elementi con la classe "box"
var boxes = document.getElementsByClassName("box");

// modifica il CSS "border" di tutti gli elementi selezionati
for (var i = 0; i < boxes.length; i++) {
  boxes[i].style.border = "1px solid black";
}

// seleziona gli elementi p figli diretti di un elemento con l'ID "main"
var paragraphs = document.querySelectorAll("#main > p");

// modifica il CSS "font-size" di tutti gli elementi selezionati
for (var i = 0; i < paragraphs.length; i++) {
  paragraphs[i].style.fontSize = "16px";
}
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. JAVASCRIPT HTML DOM

Per manipolare il contenuto di un elemento HTML utilizzando la JavaScript HTML DOM, ci sono alcune proprietà principali che si possono utilizzare: innerHTML, textContent e value.

- **innerHTML**: questa proprietà restituisce o imposta il contenuto HTML di un elemento, compresi eventuali tag HTML al suo interno

```
var elemento = document.getElementById("idElem");
elemento.innerHTML = "<p>new HTML content </p>";
```

- **textContent**: questa proprietà restituisce o imposta il testo contenuto in un elemento, ignorando eventuali tag HTML al suo interno

```
var elemento = document.getElementById("idElem");
elemento.textContent = "new text content ";
```

- **value**: questa proprietà restituisce o imposta il valore di un elemento input, come ad esempio una casella di testo o una casella di selezione

```
var input = document.getElementById("idInput");
var valoreInput = input.value;
input.value = "Nuovo valore";
```

La scelta tra le proprietà innerHTML, textContent e value dipende dal tipo di elemento HTML che si sta manipolando e dal tipo di contenuto che si sta cercando di impostare:

- La proprietà **innerHTML** è utile quando si vuole impostare un contenuto HTML che può includere tag HTML al suo interno; ad esempio, quando si vuole creare un nuovo elemento HTML da inserire nella pagina web, come una lista di elementi o una tabella.
- La proprietà **textContent** è utile quando si vuole impostare un contenuto testuale di un elemento HTML e si vuole evitare di includere tag HTML al suo interno; ad esempio, quando si vuole aggiornare il contenuto di un paragrafo o di una etichetta di un form.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- La proprietà **value** è utilizzata principalmente per gli elementi input e textarea, per impostare e recuperare il valore immesso dall'utente, come ad esempio un indirizzo email o un testo di ricerca.

In generale, è sempre buona norma utilizzare la proprietà più specifica per il tipo di elemento HTML che si sta manipolando: se si sta manipolando un elemento HTML che contiene solo testo, ad esempio, è meglio utilizzare textContent invece di innerHTML. D'altra parte, se si sta creando un nuovo elemento HTML con tag HTML specifici, allora innerHTML è la scelta migliore.

Per creare nuovi elementi HTML utilizzando la JavaScript HTML DOM, si utilizza il metodo **createElement**. Questo metodo restituisce un nuovo oggetto Element che può essere personalizzato e aggiunto alla pagina web

```
var nuovoElemento = document.createElement("div");
```

Il parametro del metodo createElement specifica il tipo di elemento HTML da creare. Una volta creato il nuovo elemento, è possibile personalizzarlo utilizzando le proprietà e i metodi del DOM, come ad esempio:

```
nuovoElemento.textContent = "Nuovo elemento";
nuovoElemento.style.color = "red";
```

Per aggiungere il nuovo elemento alla pagina web, si utilizza il metodo **appendChild**, che aggiunge l'elemento alla fine dell'elemento padre specificato

```
var elementoPadre = document.getElementById("idElementoPadre");
elementoPadre.appendChild(nuovoElemento);
```

È anche possibile inserire l'elemento in una posizione specifica utilizzando il metodo **insertBefore**, che inserisce l'elemento prima di un elemento figlio specificato

```
var elementoFiglio = document.getElementById("idElementoFiglio");
elementoPadre.insertBefore(nuovoElemento, elementoFiglio);
```

Per rimuovere un elemento HTML dalla pagina web, si utilizza il metodo **removeChild**. Questo metodo rimuove un elemento figlio specificato dall'elemento padre corrispondente

```
var elem = document.getElementById("idElementoDaRimuovere");
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
var elementoPadre = elem.parentNode;
elementoPadre.removeChild(elem);
```

Utilizzando questi metodi, è possibile creare e personalizzare nuovi elementi HTML e aggiungerli o rimuoverli dalla pagina web in modo dinamico.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Eventi

Gli **eventi** in JavaScript rappresentano azioni o comportamenti che si verificano nella pagina web, come ad esempio un clic del mouse, la pressione di un tasto sulla tastiera o il completamento di un caricamento di una pagina.

I **listener** degli eventi in JavaScript permettono di gestire in modo personalizzato gli eventi. In particolare, il metodo addEventListener() permette di aggiungere listener agli eventi. In questo modo, si può eseguire una funzione di callback specifica quando si verifica un evento.

Gli **attributi** HTML, come onclick, possono essere utilizzati per gestire alcuni eventi predefiniti come il click del mouse. Tuttavia, l'utilizzo di questi attributi HTML è considerato obsoleto e non consigliato. Il metodo addEventListener() è il metodo preferito per gestire gli eventi in JavaScript, in quanto offre maggiore flessibilità e separa la logica del comportamento dalla struttura del codice HTML.

Quindi riassumendo: gli eventi sono personalizzabili tramite listener, mentre gli attributi HTML gestiscono un set di eventi predefiniti.

Esiste una lista di nomi di eventi standardizzati che si possono utilizzare con il metodo addEventListener(). Questi nomi di eventi rappresentano azioni o comportamenti che si verificano nella pagina web, come ad esempio un clic del mouse, la pressione di un tasto sulla tastiera o il completamento di un caricamento di una pagina.

Ecco una lista di alcuni nomi di eventi comuni:

- **click**: quando un elemento viene cliccato
- **dblclick**: quando un elemento viene cliccato due volte
- **mousedown**: quando un pulsante del mouse viene premuto su un elemento
- **mouseup**: quando un pulsante del mouse viene rilasciato su un elemento
- **mouseenter**: quando il cursore del mouse entra in un elemento
- **mouseleave**: quando il cursore del mouse esce da un elemento

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **keydown**: quando viene premuto un tasto sulla tastiera
- **keyup**: quando un tasto sulla tastiera viene rilasciato
- **submit**: quando viene inviato un form
- **load**: quando una risorsa viene caricata (come un'immagine o una pagina web)
- **resize**: quando la dimensione della finestra del browser viene modificata

Per una lista completa di eventi standardizzati, si può fare riferimento alla documentazione ufficiale di MDN Web Docs all'indirizzo:

https://developer.mozilla.org/it/docs/Web/Events.

È inoltre possibile creare anche eventi personalizzati, utilizzando il metodo CustomEvent(). In questo modo, si può creare un evento personalizzato e assegnargli un nome personalizzato e specificare la logica da eseguire quando si verifica l'evento.

Ad esempio, il seguente codice aggiunge un listener degli eventi al pulsante di una pagina web e definisce una funzione da eseguire quando l'utente fa clic sul pulsante:

```
<!DOCTYPE html>
<html>
<head>
     <title>Esempio di listener degli eventi</title>
</head>
<body>
     <button id="btn">Fai clic qui</button>

     <script>
     // Aggiungiamo un listener al click del pulsante
     document.getElementById("btn").addEventListener("click",
function() {
               alert("Hai fatto clic sul pulsante!");
     });
     </script>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
</body>
</html>
```

Nel seguente esempio, creiamo una lista non ordinata (ul) dinamicamente utilizzando il metodo createElement e la aggiungiamo alla pagina utilizzando il metodo appendChild:

```
<!DOCTYPE html>
<html>
<head>
     <title>Esempio di creazione lista con JavaScript</title>
</head>
<body>
     <button id="btnAggiungi">Aggiungi elemento alla lista</button>
     <ul id="lista"></ul>

     <script>
     // Aggiungiamo un listener all'evento click del pulsante
     document.getElementById("btnAggiungi").
     addEventListener("click", function() {
          // Creiamo un nuovo elemento li
          var nuovoElemento = document.createElement("li");
          // Impostiamo il contenuto del nuovo elemento
          nuovoElemento.textContent = "Nuovo elemento";
          // Aggiungiamo il nuovo elemento alla lista

     document.getElementById("lista").appendChild(nuovoElemento);
     });
     </script>
</body>
</html>
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, abbiamo un pulsante con id **btnAggiungi** e un elemento ul con id lista. Quando l'utente fa clic sul pulsante, il codice JavaScript crea un nuovo elemento li, imposta il suo contenuto e lo aggiunge alla lista tramite il metodo **appendChild**.

È importante notare che, in questo esempio, stiamo aggiungendo gli elementi alla fine della lista. Se volessimo aggiungere gli elementi in una posizione specifica, dovremmo utilizzare il metodo **insertBefore** al posto di **appendChild**.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. *Introduzione alle Tecnologie Web*, McGraw-Hill, 2010
- A. S. Tanembaum. *Computer Networks* (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. *Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR*, 2005
- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).